CS 3006
Parallel and Distributed Computing
Lecture 9

Danyal Farhat

FAST School of Computing

NUCES Lahore

Programming Shared Address Space Platforms using POSIX Thread API

Lecture's Agenda

- **Thread**
- **Multithreading**
- **Thread vs. Multithreading**
- **Pthreads API**
- **Pthread_Create**
- **Joining and Detaching Threads**
- **Thread Example**
- **Synchronization Problem**

Lecture's Agenda (Cont.)

- **Mutual Exclusion**
  - **Critical Section**
- **Locking Overhead**
  - **Trylock()**
- **Producer Consumer Using Locks**
- **Condition Variables for Synchronization**
- **Producer-Consumer Problem**
- **Summary**
- **Additional Resources**

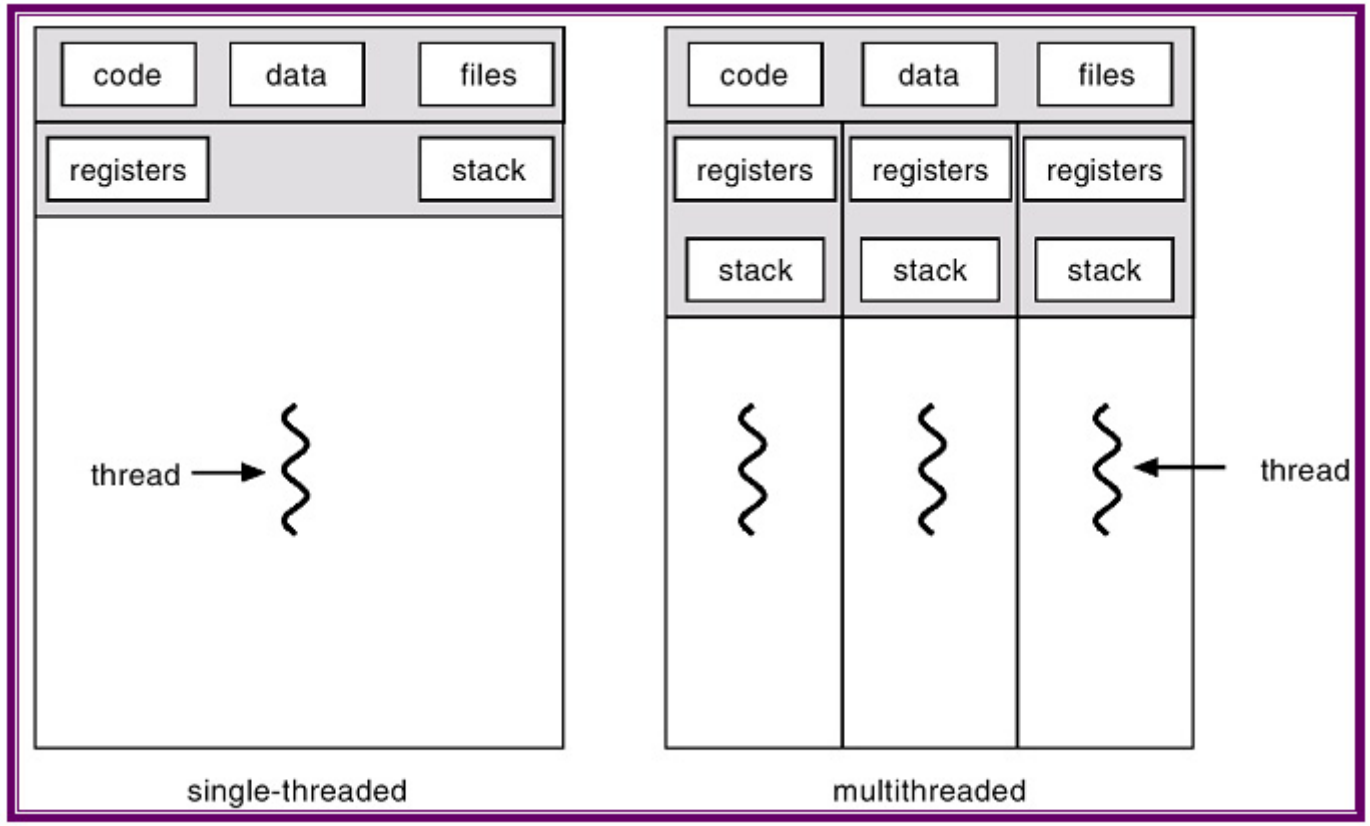
Thread

- A thread is “an independent stream of instructions that can be scheduled to run by the operating system”
- A thread is also a “procedure that runs independently from its main program”
- Pthreads have been specified (for UNIX) by IEEE POSIX 1003.1c standard (1995)
- There exist other threads libraries, such as Java threads, Solaris threads etc.

Multithreading

- **Operating system facility that enables an application to create multiple threads of execution within a process**
- **Many different users can run programs that appears to be running at the same time**
- **However with single processing unit, they are not running at exact same time**
- **Operating system switches available resources from one running program to another**
- **Multiple threads exist within each process and share resources like memory**

Thread vs. Multithreading



Pthreads API

- **POSIX thread API**
  - **Portable Operating System Interface (POSIX)**
  - **standard threads API, supported by most vendors**

- **Pthreads are interesting for:**
  - **Overlapping I/O and CPU work; some threads can block for I/O while others can continue**
  - **Scheduling and load balancing**
  - **Ease of programming and widespread use**
  - **In parallel programming they can be very useful, since communications between threads are much faster (3-5 times)**

Pthreads API (Cont.)

- **Thread management**
  - **Create, detach, join, thread attributes**
- **Mutexes**
  - **Mutual exclusion, create, destroy, lock, unlock, mutex attributes**
- **Condition variables**
  - **Create, destroy, wait, signal, programmer specified conditions, condition variable attributes**
- **pthreads.h header file**
- **Pthreads are defined for C; some FORTRAN compilers also have Pthreads API (e.g. IBM AIX)**
- **Pthreads API are based on over 60 calls pthread_* ;**

Pthread_Create

int pthread_create (
pthread_t *thread_handle,
const pthread_attr_t *attribute,
void * (*thread_function)(void *),
void *arg);

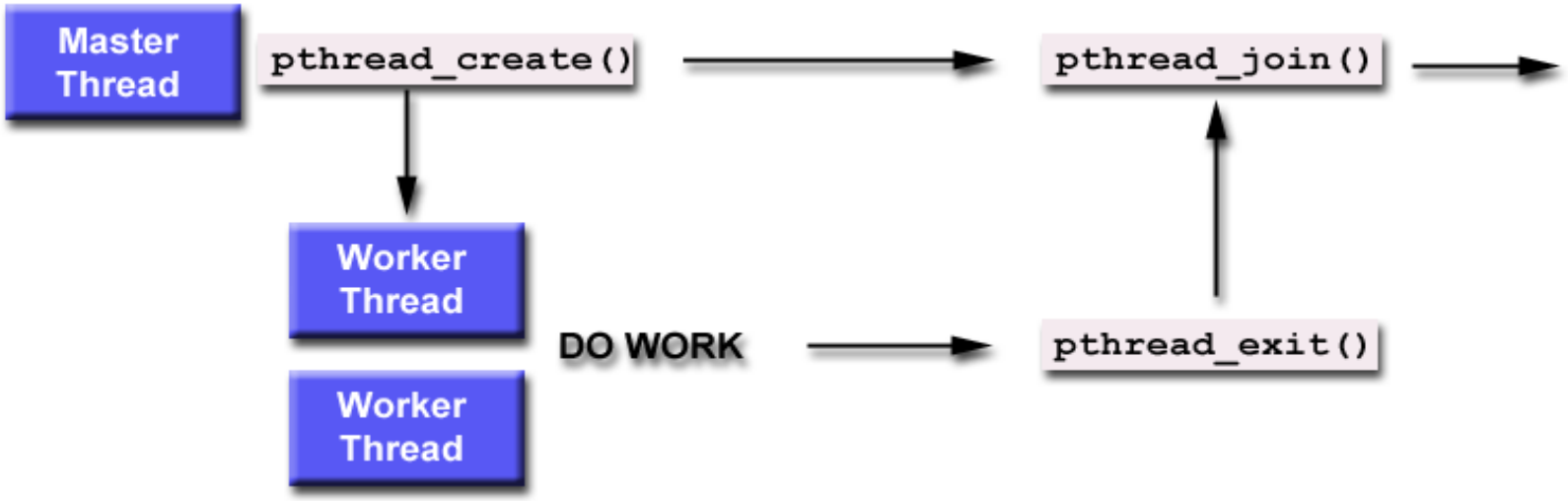
- **thread_handle unique identifier**
- **attribute NULL for default attributes**
- **thread_function C routine executed once thread is created**
- **arg a single argument that may be passed to thread_function; NULL for no argument**

- **It can be called any number of times, from anywhere, there is no hierarchy or dependency**

Joining and Detaching Threads

- **int pthread_join(pthread_t thread_handle,**
  void **value_ptr)
- **It is possible to get return status if specified in pthread_exit()**
- **Only one pthread_join() call can be matched**
- **Thread can be joinable or detached (no possibility to join); it is better to declare it for portability!**
- **Int pthread_detach(pthread_t thread); is used to indicate to the implementation that storage for the thread can be reclaimed when the thread terminates.**
- **If thread has not terminated, pthread_detach() will not cause it to terminate. It works even if thread was created as joinable**

Joining and Detaching Threads (Cont.)



- **POSIX standard does not specify stack size for a thread; exceeding the limit produces Segmentation fault**
- **Safe and portable programs explicitly allocate enough stack**

Thread Example

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

void *PrintHello(void *threadid)
{
        long tid;
        tid = (long)threadid;
        printf("Hello World! It's me, thread #%ld!\n", tid);
pthread_exit(NULL);
 }
```

Thread Example (Cont.)

```
int main (int argc, char *argv[])
{
        pthread_t threads[NUM_THREADS];
        int  long t;
        for(t=0; t<NUM_THREADS; t++)
       {
                pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        }
        exit(-1);
}
```

Process Synchronization Problem

- **Example: Bank Transactions**
  - **Current balance = Rs. 70,000**
  - **Check deposited = Rs. 10,000**
  - **ATM withdrawn = Rs. 5,000**
- **Correct Balance after both transactions**
  - **Balance = 75000**
- **What if both transactions are initiated at the same time!**
- **Check Deposit:**
  - **MOV A, Balance // A = 70,000**
  - **ADD A, Deposited // A = 80,000**
- **ATM Withdrawal:**
  - **MOV B, Balance // B = 70,000**
  - **SUB B, Withdrawn // B = 65,000**

Mutual Exclusion

- **Mutual exclusion (Mutex) variables work like a lock protecting access to a shared resource**
- **Only one thread can lock a mutex at a moment; even if more than one thread tries to lock the mutex, only one will be successful; this avoids race**
- **Sequence:**
  - **Creation of the mutex**
  - **More than one thread tries to lock the mutex**
  - **Only one locks it**
  - **The owner makes changes**
  - **The owner unlocks it**
  - **Another thread gets mutex (it was blocked, unblocking is automatic) and the process repeats**
  - **At the end mutex is destroyed**

Critical Section

```
do
{
    Entry section

      critical section

    Exit section

   remainder section

} while(1)
```

Mutual Exclusion (Cont.)

- **The Pthreads API provides the following functions for handling mutex-locks:**

```
int pthread_mutex_init ( pthread_mutex_t           *mutex_lock,
const pthread_mutexattr_t *lock_attr);

int pthread_mutex_lock (pthread_mutex_t *mutex_lock);

int pthread_mutex_unlock (pthread_mutex_t *mutex_lock);
```

- **"init" function initializes the mutex-lock to an unlocked state.**

Mutual Exclusion (Cont.)

For example:

```
pthread_mutex_t total_cost_lock;
...
main() {
        ....
        pthread_mutex_init(&total_cost_lock, NULL);
        ....
}
void *add_cost(void *costn) {
        ....
        pthread_mutex_lock(&total_cost_lock);

        total_cost = total_cost + costn;
        /* and unlock the mutex */
        pthread_mutex_unlock(&total_cost_lock);
}
```

Locking Overhead

- Locks represent serialization points since critical sections must be executed by threads one after the other
- Encapsulating large segments of the program within locks can lead to significant performance degradation
- It is often possible to reduce the idling overhead associated with locks using an alternate function, pthread_mutex_trylock

```
int pthread_mutex_trylock ( pthread_mutex_t *mutex_lock);
```

- pthread_mutex_trylock is typically much faster than pthread_mutex_lock on typical systems since it does not have to deal with queues associated with locks for multiple threads waiting on the lock

Trylock()

```
pthread_mutex_t total_cost_lock;
Int lock_status;
main() {
        pthread_mutex_init(&total_cost_lock, NULL);
        ....
}
void *add_cost(void *costn) {
        ....
        Lock_status pthread_mutex_trylock(&total_cost_lock);
        If (lock_status == EBUSY)
            addlater;
        else
            total_cost = total_cost + costn;
            /* and unlock the mutex */
            pthread_mutex_unlock(&total_cost_lock);
}
```

Producer Consumer Using Locks

- **Multi-process synchronization problem**
- **Also known as Bounded Buffer problem**
- **Producer puts information in the buffer**
- **Consumer takes information from the buffer**

- **Example:**
  - **Multiple requests sending to the server (buffer)**
  - **Multiple printing requests sending to the printer (buffer)**

Producer Consumer Using Locks (Cont.)

```
pthread_mutex_t task_queue_lock;
int task_available;
main() {
    ....
    task_available = 0;
    pthread_mutex_init(&task_queue_lock, NULL);
}
void *producer(void *producer_thread_data) {
    ....
    while (!done()) {
        inserted = 0;
        create_task(&my_task);
        while (inserted == 0) {
            pthread_mutex_lock(&task_queue_lock);
            if (task_available == 0) {
                insert_into_queue(my_task);
                task_available = 1;
                inserted = 1;
            }
            pthread_mutex_unlock(&task_queue_lock);
        }
    }
```

Producer Consumer Using Locks (Cont.)

```
void *consumer(void *consumer_thread_data) {
   int extracted;
   struct task my_task;
   /* local data structure declarations */
   while (!done()) {
      extracted = 0;
      while (extracted == 0) {
         pthread_mutex_lock(&task_queue_lock);
         if (task_available == 1) {
            extract_from_queue(&my_task);
            task_available = 0;
            extracted = 1;
         }
         pthread_mutex_unlock(&task_queue_lock);
      }
      process_task(my_task);
   }
}
```

Condition Variables for Synchronization

- **A condition variable allows a thread to block itself until specified data reaches a predefined state**

- **A condition variable is associated with this predicate. When the predicate becomes true, the condition variable is used to signal one or more threads waiting on the condition**

- **A single condition variable may be associated with more than one predicate**

Condition Variables for Synchronization (Cont.)

- **A condition variable always has a mutex associated with it. A thread locks this mutex and tests the predicate defined on the shared variable**

- **If the predicate is not true, the thread waits on the condition variable associated with the predicate using the function pthread_cond_wait.**

Condition Variables for Synchronization (Cont.)

- **Pthreads provides the following functions for condition variables:**

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);

int pthread_cond_signal(pthread_cond_t *cond);

int pthread_cond_broadcast(pthread_cond_t *cond);

int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);

int pthread_cond_destroy(pthread_cond_t *cond);
```

Producer-Consumer Problem

```
pthread_cond_t cond_queue_empty, cond_queue_full;
pthread_mutex_t task_queue_cond_lock;
int task_available;


main() {

   task_available = 0;
   pthread_init();
   pthread_cond_init(&cond_queue_empty, NULL);
   pthread_cond_init(&cond_queue_full, NULL);
   pthread_mutex_init(&task_queue_cond_lock, NULL);
   /* create and join producer and consumer threads */
}
```

Producer

```
void *producer(void *producer_thread_data) {
   int inserted;
   while (!done()) {
      create_task();
      pthread_mutex_lock(&task_queue_cond_lock);
      while (task_available == 1)
         pthread_cond_wait(&cond_queue_empty,
            &task_queue_cond_lock);
      insert_into_queue();
      task_available = 1;
      pthread_cond_signal(&cond_queue_full);
      pthread_mutex_unlock(&task_queue_cond_lock);
   }
}
```

Consumer

```
void *consumer(void *consumer_thread_data) {
      while (!done()) {
            pthread_mutex_lock(&task_queue_cond_lock);
            while (task_available == 0)
                  pthread_cond_wait(&cond_queue_full,

                  &task_queue_cond_lock);
              my_task = extract_from_queue();
              task_available = 0;
            pthread_cond_signal(&cond_queue_empty);
            pthread_mutex_unlock(&task_queue_cond_lock);
            process_task(my_task);
      }
 }
```

Summary

- **Thread**
  - **A procedure that runs independently from its main program**
- **Multithreading**
  - **OS facility that enables an application to create multiple threads of execution within a process**
- **Thread vs. Multithreading**
- **Pthreads API**
  - **Portable Operating System Interface library useful for overlapping I/O and CPU work**

Summary (Cont.)

- **Pthread_Create**
  - **int pthread_create (pthread_t *thread_handle, const pthread_attr_t *attribute, void * (*thread_function)(void *), void *arg);**
- **Joining and Detaching Threads**
  - **int pthread_join (pthread_t thread_handle, void **value_ptr)**
- **Thread Example**
- **Synchronization Problem**
- **Mutual Exclusion**
  - **Mutual exclusion (Mutex) variables work like a lock protecting access to a shared resource**

Summary (Cont.)

- **Critical Section**
- **Locking Overhead**
  - **Trylock()**
- **Producer Consumer Using Locks**
  - **Also known as Bounded Buffer problem**
  - **Producer puts information in the buffer**
  - **Consumer takes information from the buffer**
- **Condition Variables for Synchronization**
- **Producer-Consumer Problem**

Additional Resources

- **Introduction to Parallel Computing by Ananth Grama and Anshul Gupta**

  - **Chapter 7. Programming Shared Address Space Platforms**

Questions?

Class Activity